COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 15, 2026
NAME:	RALPH FLORENZ DUE?... ANDREW DOLOSO JR.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILOA.

Screenshot

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis:

Doloso:

- This code demonstrates how tasks are formally arranged within a fixed queue, ensuring there is no overlapping during execution.

Duenas

- In the code given, we have a callable function and we used the "threads" standard library to turn it into a thread. By doing this, we can simultaneously call it with other threads.

Table 13-1. Simple One-Threaded Example

ILOB.

Screenshot

```cpp
#include <iostream>
#include <string>
#include <thread>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

```
1 : Competent0 : T.I.P.
2 : Computer
3 : Engineers
```

Analysis:

Doloso:

- This code shows a random output order. Every time you run the code, it gives a different result because it is multi-threaded.

Duenas:

- Now for Part 2, we implemented multithreading. We made 2 vectors, one that consists of

	string values, and one consists of threads. The for loop "pushes" the vector of strings with their corresponding number which is "i" to the vector of threads in this format; "Number: String". After that, we made an alias for each element in the threads, and when using the ".join()" we printed all of the elements simultaneously. This is the reason why the output is not consistent in every run.

Table 13-2. Multithreaded Example.

2. Supplementary Activity

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages

Doloso:
- Multithreading lets multiple tasks run at the same time using concurrency and parallelism, which cuts down execution time. The downside is that since tasks run concurrently, the output order can be unpredictable on every run.

Duenas:
- Multithreading in programming is a technique that implements concurrency and parallelism. It is when two or more tasks are working simultaneously on a single or different data. The advantages of this is that it cuts the time it takes for the compiler to read through individual functions by executing them at the same time. In return, the disadvantage of this is that data may be returned in different orders every run due to the nature of the compiler when giving the output per line.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Doloso:
- Web Browsers: One thread loads scripts or assets while another keeps the UI and tabs responsive. Media Players: Background threads buffer and stream video/audio while a primary thread handles playback. Text Editors / IDEs: Background threads run syntax checking, auto-save, and spellcheck without lagging your typing.

Duenas:
- Multithreading can be useful when: • A system programmer wants to cut time in computation. • A data engineer makes a program to divide the large amount of data into subparts and simultaneously work on the subgroups of data. • A calculator manufacturer makes a calculator that can process different computations at the same time, depending on the user.

3. Differentiate between parallelism and concurrency.

Doloso:

- Concurrency is about managing multiple tasks at the same time by switching between them, while parallelism is about physically executing multiple tasks at the exact same time using separate CPU cores.

Duenas:

- Concurrency occurs when two or more tasks are working on a single data. Parallelism occurs when two or more tasks are working on individual data.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that:

o Display
o T1.join()
o Display
o T2.join()
o Display
o T3.join()

- Provide an analysis based on the outputs of the multi-threading exercise.

CODE OUTPUT

Welcome | partB.cpp | main_C.cpp | partC.h

SUPPLEMENTARY > partB.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <chrono>

int integer;

void addInteger(int i) {
    integer += i;
}

void display() {
    std::cout << "integer: " << integer << "\n";
}

int main() {
    std::thread T1(addInteger, 1);
    std::thread T2(addInteger, 2);
    std::thread T3(addInteger, 3);

    display();
    T1.join();
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    display();
    T2.join();
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    display();
    T3.join();

    return 0;
}
```

Problems | Output | Debug Console | Terminal

```
PS C:\Users\Doloso\Documents\CPE010_DOL
integer: 0
integer: 6
integer: 6
PS C:\Users\Doloso\Documents\CPE010_DOL
```

Analysis
Doloso: - The early output varies because of a race condition: the main thread reads the global variable while worker threads are still updating it. Getting a 0 or 1 is rare because the initial setup delay for new threads usually gives them enough time to finish before the main thread prints. Once join() is called, it forces the main thread to wait for every thread to finish, making the final result of 6 completely consistent. Duenas: For the applied program from the problem, it shows mostly these outputs. We only added the chrono standard library for the sake of observation. To put my analysis simply, the first display shows us that sometimes the global integer has already been changed to either the sum of 1 and 2 or 1 and 3. In rare cases it will show just 1 or 0 (which I have not attained). Why is this considered rare? Based on my understanding of threads, right when the compiler reads the line of where you defined the thread, it will start the function immediately. As to why the first display of the global integer is always different is because when the compiler reads the main, it first executes the parameter 1 and after that parameter 2 and lastly parameter 3. How fast these functions finish is purely dependent on the hardware, getting only 1 at the display means that it has finished faster than the other threads, but most of the time this doesn't happen. The two other displays display 6 because by the time T2.join() has been called, the three threads already finished their function.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

CODE
MAIN_C.CPP FILE

Welcome | partB.cpp | main_C.cpp | partC.h

SUPPLEMENTARY > Part_C > main_C.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include "partC.h"

template <typename T>
void display(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
}

int main() {
    int arr1[15] = {42, 7, 19, 3, 55, 12, 88, 24, 1, 33, 67, 14, 9, 50, 21};
    int arr2[15] = {99, 11, 45, 2, 78, 34, 13, 60, 5, 82, 18, 27, 51, 8, 36};

    std::thread T1(insertionSort<int>, arr1, 15);
    std::thread T2(insertionSort<int>, arr2, 15);

    T1.join();
    T2.join();

    display(arr1, 15);
    std::cout << std::endl;
    display(arr2, 15);
    std::cout << std::endl;

    return 0;
}
```

PARTC.H FILE

Welcome | partB.cpp | main_C.cpp | partC.h

SUPPLEMENTARY > Part_C > partC.h > MAIN_C_H

```cpp
#ifndef MAIN_C_H
#define MAIN_C_H

template <typename T>
void insertionSort(T arr[], int size) {
    int k = 1, j;
    T temp;

    while (k < size) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp <= arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}

#endif
```

OUTPUT

```
PS C:\Users\Doloso\Documents\CPE010_DOLOSO-CPE21S1\14P1> cd "c:\Users\Doloso\Do
\main_C }
1 3 7 9 12 14 19 21 24 33 42 50 55 67 88
2 5 8 11 13 18 27 34 36 45 51 60 78 82 99
PS C:\Users\Doloso\Documents\CPE010_DOLOSO-CPE21S1\14P1\SUPPLEMENTARY\Part_C>
```

Analysis

Doloso:

- He reused our insertion sort algorithm to test how two threads handle sorting in parallel. By assigning each array to its own thread, the CPU could run both sorts at the same time without memory conflicts. Altwain noticed that small arrays sort so quickly that worker threads often finish before the main thread reaches the print statement. This taught us that display placement is key: printing before join() catches the arrays mid-sort, while printing after join() guarantees fully sorted results.

Duenas:

- For this code, I reused one of my submitted sorting algorithms, specifically the insertion sort. In the main driver function, the program declared two arrays with unsorted values and then made two threads that sorted these values with the insertion sort algorithm, thus starting their sorting. By the time we've displayed the arrays, the values have already been sorted. It is where we place the display code when

we can see different outputs. The race condition was demonstrated in the Procedure, where each outcome is slightly different from the other. And the supplementary activity shows how the .join() actually works.

CONCLUSION/LEARNING ANALYSIS

Doloso:

- This activity really showed me how multithreading trades strict control for speed. While running tasks simultaneously speeds up performance, sharing data without proper synchronization leads to unpredictable race conditions. I learned that using tools like join() is crucial because it gives you control back when you need it, ensuring all threads finish before you try to use or display their final results.

Duenas:

- In this activity, we learned a lot on how multithreading works, what are the key factors, and its processing mechanics. A condition called the "race condition" is basically how printing the result of two threads is not always the same. The compiler only prints per line, so every function has a speed at which it processes and it is entirely dependent on the hardware. I need to improve my understanding on multithreading so I can fully utilize this in future works.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: *QuillBot*

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.